

ECF — TITRE PROFESSIONNEL DÉVELOPPEUR WEB ET WEB MOBILE

Vite & Gourmand

Documentation technique

Auteur	Nabil Lachemi
Application en ligne	https://vite-gourmand-tp-luo4.onrender.com
Dépôt GitHub	https://github.com/nabil77130/Vite-Gourmand-TP
Stack	PHP 8.2 · Symfony 6.4 LTS · SQLite · MongoDB Atlas · Docker / Render
Version du document	23 septembre 2026

Sommaire

1. Réflexions initiales et choix technologiques — analyse du besoin, stack retenue, alternatives

2. Configuration de l'environnement de travail — outils, installation, variables d'environnement

3. Architecture de l'application — organisation MVC, services, routes

4. Modèle de données — MCD, tables, base NoSQL

5. Diagramme de cas d'utilisation

6. Diagramme de séquence — passer une commande

7. Règles de gestion — prix, stock, statuts, emails

8. Sécurité

9. Accessibilité (RGAA)

10. Déploiement — démarche et étapes

11. Tests et vérifications

1. Réflexions initiales et choix technologiques

1.1 Analyse du besoin

Vite & Gourmand est un traiteur bordelais de deux personnes, Julie et José, qui envoie jusqu'ici ses menus par email à ses habitués. L'application doit :

- **présenter** l'entreprise et ses menus à tous les visiteurs, avec des filtres rapides ;
- permettre de **commander** un menu en ligne, avec un prix calculé selon des règles précises (minimum de convives, réduction, livraison) ;
- donner à l'équipe un **espace de gestion** : menus, plats, horaires, commandes et leurs statuts, avis clients ;
- offrir à l'administrateur des **statistiques** issues d'une base **non relationnelle**.

Contraintes imposées : une base relationnelle et une base NoSQL, une application **déployée et fonctionnelle**, **accessible** (RGAA) et **sécurisée**. Les utilisateurs finaux (Julie et José) n'ont pas de compétences informatiques : l'interface d'administration doit rester simple.

1.2 Stack retenue et justification

Besoin	Choix	Justification
Langage / framework back-end	PHP 8.2 + Symfony 6.4 LTS	Version à support long terme. Symfony fournit en standard ce qu'il faudrait sinon écrire à la main avec PHP et PDO : authentification, hachage des mots de passe, protection CSRF, validation des formulaires, gestion des rôles. Moins de code « maison », donc moins de failles.
Accès aux données SQL	Doctrine ORM + migrations	Requêtes paramétrées (protection contre l'injection SQL), schéma versionné par migrations, et indépendance vis-à-vis du SGBD : passer de SQLite à MySQL ou PostgreSQL ne demande que de changer <code>DATABASE_URL</code> .
Base relationnelle	SQLite	Un seul fichier, aucun serveur à installer : idéal pour un projet de cette taille et pour le déploiement. Limite assumée : peu adaptée aux écritures concurrentes massives ; en production réelle, on passerait à PostgreSQL.
Base NoSQL	MongoDB Atlas (bibliothèque <code>mongodb/mongodb</code>)	Les statistiques sont une lecture analytique, séparée de la base transactionnelle. Une copie dénormalisée de chaque commande permet des agrégations directes (<code>\$group</code> , <code>\$sum</code>) sans jointure. Atlas offre un cluster gratuit hébergé.
Front-end	Twig, CSS, JavaScript, Symfony UX Turbo, AssetMapper	Rendu côté serveur (bon référencement, fonctionne sans JavaScript). Turbo met à jour la liste des menus filtrés <i>sans rechargement</i> , comme demandé. AssetMapper évite toute chaîne Node.js/Webpack.
Emails	Symfony Mailer + Mailtrap	Envoi SMTP standard ; Mailtrap intercepte les emails en test, sans risque d'écrire à de vrais clients.
Déploiement	Docker sur Render	Offre gratuite sans carte bancaire acceptant les images Docker ; le Dockerfile installe lui-même l'extension MongoDB, qui manque chez la plupart des hébergeurs mutualisés.

1.3 Alternatives étudiées

Alternative	Raison de l'écarter
PHP natif avec PDO	Possible, mais tout le socle de sécurité (CSRF, sessions, rôles, validation) aurait été à écrire et à tester à la main.
MySQL / MariaDB	Excellent choix en production, mais demande un serveur de base de données hébergé ; SQLite suffit ici, et Doctrine permet de migrer plus tard.
MongoDB ODM (Doctrine)	Couche supplémentaire inutile pour une seule collection de statistiques ; la bibliothèque officielle suffit.
React / Vue en front	Complexité (API, build JavaScript) disproportionnée ; Turbo apporte l'interactivité demandée avec du rendu serveur.
Heroku, fly.io, alwaysdata	Heroku n'a plus d'offre gratuite ; fly.io demande une carte bancaire ; l'offre gratuite d'alwaysdata interdit les connexions à une base distante (MongoDB Atlas).

2. Configuration de l'environnement de travail

2.1 Outils

Outil	Rôle
Windows + XAMPP	Fournit PHP 8.2 (thread-safe, x64) en local.
Composer 2	Installation des dépendances PHP (<code>composer.json</code> / <code>composer.lock</code>).
Visual Studio Code	Éditeur, avec terminal intégré (PowerShell) ; Cmdr comme terminal alternatif.
Git + GitHub	Versionnement, branches <code>main</code> / <code>develop</code> / <code>feature/*</code> .
Extension PHP mongodb	DLL PECL correspondant à la version de PHP, déclarée dans <code>php.ini</code> (<code>extension=mongodb</code>).
MongoDB Atlas	Cluster gratuit hébergé ; accès réseau et utilisateur de base configurés dans l'interface Atlas.
Mailtrap	Boîte email de test (SMTP sandbox).
Serveur PHP intégré	<code>php -S 127.0.0.1:8000 -t public</code> pour le développement.
Chrome + outils de développement	Tests d'affichage (ordinateur et mobile), console, réseau, contrastes.

2.2 Installation du projet en local

```
git clone https://github.com/nabil77130/Vite-Gourmand-TP.git
cd Vite-Gourmand-TP
composer install
# créer .env.local (voir 2.3), puis :
php bin/console doctrine:migrations:migrate
php bin/console doctrine:fixtures:load          # données de démonstration
php bin/console app:sync-order-stats --purge    # copie des commandes vers MongoDB
php bin/console asset-map:compile              # CSS et JavaScript
php -S 127.0.0.1:8000 -t public
```

Les fichiers `sql/schema.sql` et `sql/data.sql` permettent aussi de créer la base directement en SQL : `sqlite3 var/data.db < sql/schema.sql` puis `sqlite3 var/data.db < sql/data.sql` .

2.3 Variables d'environnement et secrets

Le fichier `.env` , versionné, ne contient que des valeurs par défaut **sans secret**. Les identifiants réels sont placés dans `.env.local` , exclu de Git par `.gitignore` .

Variable	Contenu
<code>APP_ENV</code> / <code>APP_SECRET</code>	Environnement (<code>dev</code> / <code>prod</code>) et clé secrète de Symfony (sessions, jetons CSRF).
<code>DATABASE_URL</code>	<code>sqlite:///kernel.project_dir%/var/data.db</code>
<code>MAILER_DSN</code>	Connexion SMTP Mailtrap (<code>null://null</code> pour désactiver l'envoi).

MONGODB_URI /
MONGODB_DB

Chaîne de connexion Atlas et nom de la base.

Ces variables sont injectées dans les services par `config/services.yaml` (`%env(MONGODB_URI)%`) : aucun identifiant n'apparaît dans le code source.

3. Architecture de l'application

L'application suit le modèle **MVC** de Symfony : les *contrôleurs* reçoivent la requête, s'appuient sur les *entités* Doctrine et les *services* métier, puis rendent une *vue* Twig.

```
src/
  Controller/           pages publiques, commande, espace client
  Controller/Admin/     espace employé et administrateur (préfixe /admin)
  Entity/              12 entités Doctrine (voir MCD)
  Form/                formulaires Symfony et leurs contraintes de validation
  Repository/          requêtes (filtres de la carte, recherche de commandes)
  Service/OrderPricer.php      calcul du prix d'une commande
  Service/OrderStatsRepository.php  accès à MongoDB (écriture + agrégations)
  Security/UserChecker.php      refuse la connexion d'un compte désactivé
  Validator/PasswordPolicy.php  politique de mot de passe centralisée
  Command/SyncOrderStatsCommand.php  resynchronisation SQL -> MongoDB
templates/            vues Twig (base, pages, emails)
docker/              Apache, PHP, script de démarrage, diagnostic
sql/                 schema.sql et data.sql
```

3.1 Services métier

- **OrderPricer** : applique les règles de prix (section 7). Il est appelé à la création, à la modification et par l'aperçu du prix en JavaScript : le prix affiché avant validation est donc exactement celui qui sera facturé.
- **OrderStatsRepository** : écrit une copie de chaque commande dans MongoDB (*upsert* idempotent sur l'identifiant de commande) et fournit les agrégations du tableau de bord.
- **UserChecker** et **PasswordPolicy** : voir section 8.

3.2 Principales routes

Route	Accès	Rôle
/, /menu, /menu/{id}	public	Accueil, carte filtrable, fiche détaillée
/register, /login, /mot-de-passe-oublie	public	Compte, connexion, réinitialisation
/contact, /mentions-legales, /cgv	public	Contact et pages légales
/order/create/{menuId}, /order/{id}/edit, /order/{id}/cancel	ROLE_USER	Commande, modification, annulation
/order/price/{menuId}	ROLE_USER	Aperçu du prix (JSON)
/user/orders, /user/profile	ROLE_USER	Suivi des commandes, profil
/admin/menu, /admin/product, /admin/horaire, /admin/order, /admin/review	ROLE_EMPLOYEE	Gestion quotidienne
/admin/employee, /admin/dashboard	ROLE_ADMIN	Comptes employés, statistiques MongoDB

4. Modèle de données

4.1 Modèle conceptuel (base relationnelle)

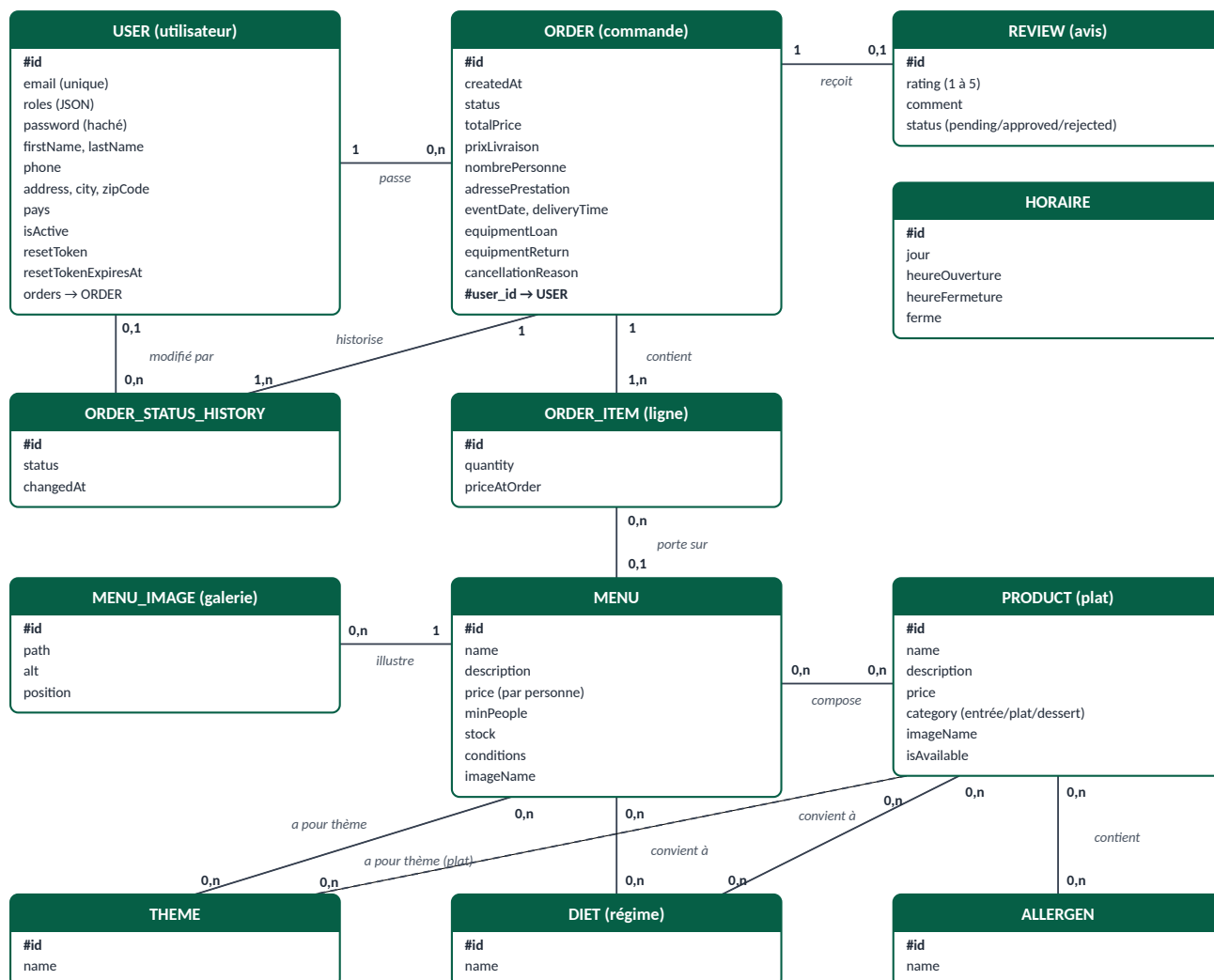


Figure 1 — Modèle conceptuel de données (cardinalités de type Merise ; # = identifiant ou clé étrangère)

Les associations « plusieurs à plusieurs » deviennent des **tables de liaison** : `menu_product`, `menu_theme`, `menu_diet`, `product_allergen`, `product_diet`, `product_theme`. Une entrée ou un dessert peut ainsi figurer dans plusieurs menus.

4.2 Rôle des tables

Table	Contenu et points notables
<code>user</code>	Comptes. <code>email</code> unique (identifiant de connexion), <code>roles</code> en JSON, mot de passe haché, <code>is_active</code> pour désactiver un employé, jeton de réinitialisation avec date d'expiration.
<code>menu</code>	Prix par personne , nombre minimum de convives, <code>stock</code> (commandes encore possibles, NULL = illimité), conditions du menu.
<code>menu_image</code>	Galerie : chemin, texte alternatif (accessibilité), ordre d'affichage.
<code>product</code>	Plats, avec catégorie (entrée, plat, dessert, boisson) ; allergènes, régimes et thèmes par tables de liaison.
<code>order</code>	Commande : prestation (adresse, date, heure), convives, livraison, total, statut courant, motif d'annulation. Nom entre guillemets en SQL (mot réservé).
<code>order_item</code>	Menu commandé et prix au moment de la commande.
<code>order_status_history</code>	Chaque changement de statut, horodaté, avec son auteur : c'est le suivi affiché au client.
<code>review</code>	Avis (note 1 à 5, commentaire), publié seulement après validation (<code>approved</code>).
<code>horaire</code>	Horaires du lundi au dimanche affichés en pied de page.

Script de création complet : `sql/schema.sql` ; jeu de données : `sql/data.sql` .

4.3 Base NoSQL (MongoDB)

Collection `order_stats` : un document autonome par commande, sans jointure nécessaire.

```
{
  "orderId": 53,           // clé de l'upsert : jamais de doublon
  "menuId": 49,
  "menuName": "Festin Italien",
  "status": "delivered",
  "nombrePersonne": 16,
  "totalPrice": 316.8,
  "prixLivraison": 0,
  "createdAt": "2026-09-21 00:18:34",
  "createdAtDay": "2026-09-21" // regroupement par jour et filtre par période
}
```

Le tableau de bord exécute un pipeline d'agrégation : `$match` (filtres par menu et par période), `$group` par menu (`$sum` du nombre de commandes et du chiffre d'affaires), puis `$sort` . Chaque création, modification ou annulation de commande met ce document à jour ; en cas d'indisponibilité de MongoDB, la commande du client n'échoue pas et l'erreur est journalisée. La commande `app:sync-order-stats` resynchronise l'ensemble.

5. Diagramme de cas d'utilisation

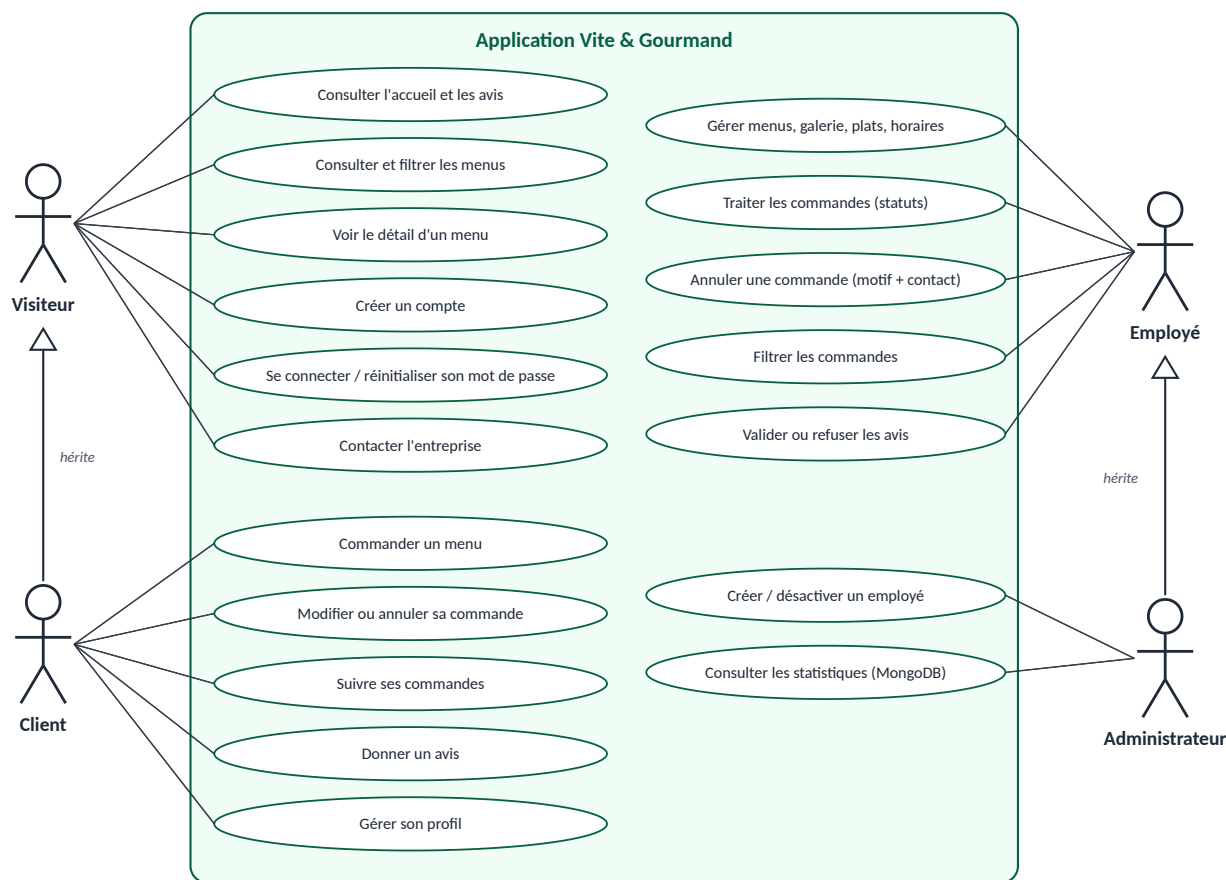


Figure 2 — Acteurs et cas d'utilisation

Le **client** hérite de tous les cas du visiteur ; l'**administrateur** hérite de tous ceux de l'employé. Ces héritages correspondent à la hiérarchie des rôles de Symfony : `ROLE_ADMIN` $\supset$ `ROLE_EMPLOYEE` $\supset$ `ROLE_USER`. Aucun compte administrateur ne peut être créé depuis l'application : le seul existe dès l'installation.

6. Diagramme de séquence : passer une commande

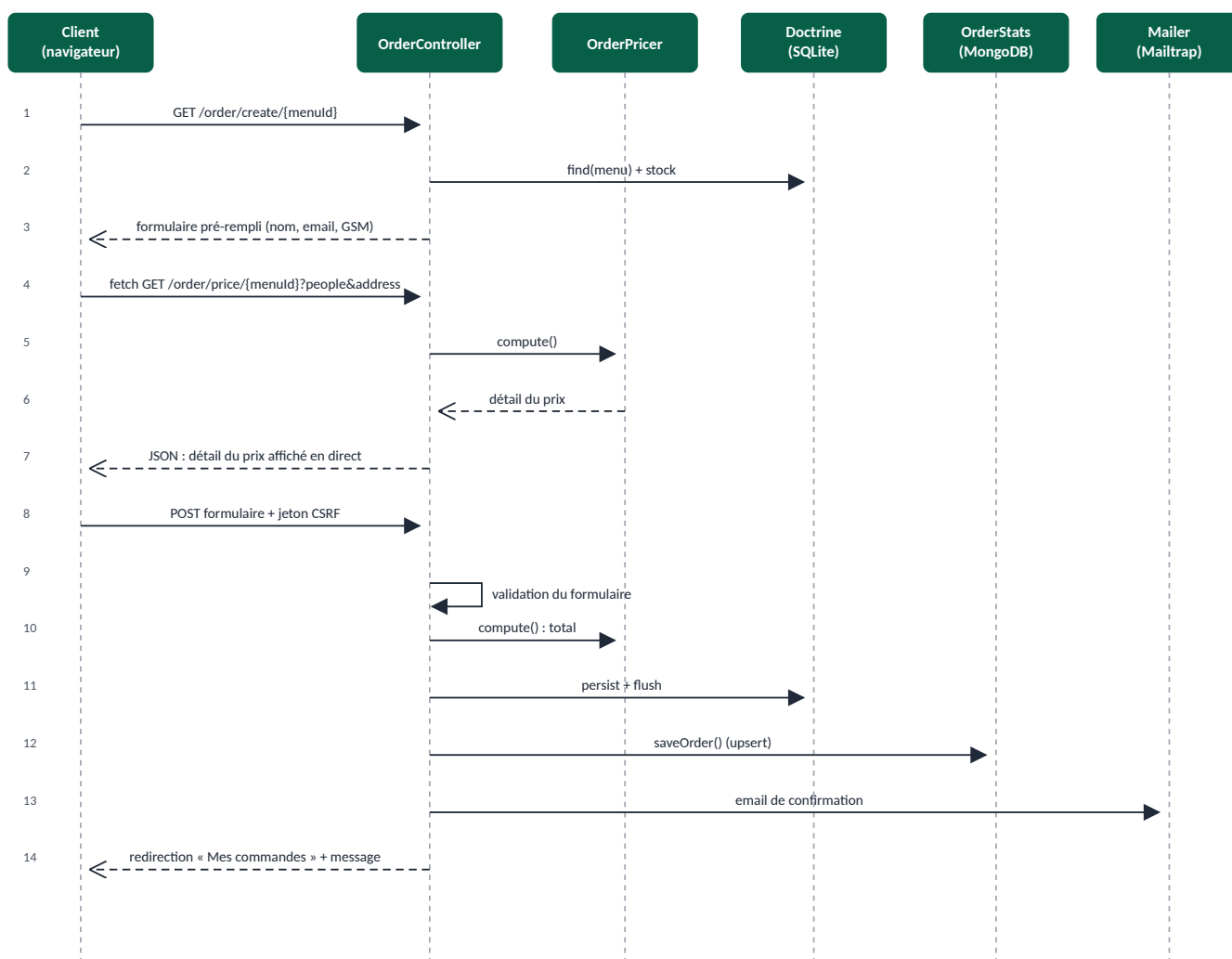


Figure 3 — Commande d'un menu par un client connecté

1. Le client ouvre la page de commande depuis la fiche du menu. Le contrôleur refuse l'accès si le menu est épuisé, même si l'adresse est saisie directement.
2. Pendant la saisie, le navigateur interroge `/order/price/{menuId}` (étapes 4 à 7) : le détail du prix est calculé **par le serveur**, avec le même service que la facturation.
3. À la validation, le formulaire est contrôlé (jeton CSRF, minimum de convives, champs obligatoires), le total est recalculé côté serveur, puis la commande, sa première entrée d'historique et la baisse du stock sont enregistrées en une transaction (`flush`).
4. La copie MongoDB et l'email de confirmation sont envoyés ensuite. Leurs éventuelles erreurs sont **journalisées sans bloquer** le client, dont la commande est déjà enregistrée.

7. Règles de gestion

7.1 Prix d'une commande (OrderPricer)

- Le menu affiche son prix **pour le nombre minimum de personnes** (prix par personne × minimum).
- Prix des repas = prix par personne × nombre de convives, avec le minimum du menu obligatoire.
- Réduction de 10 %** sur les repas dès 5 convives de plus que le minimum.
- Livraison** gratuite à Bordeaux ; ailleurs 5 € + 0,59 € par kilomètre. La distance est lue dans une table des communes de la métropole (20 km par défaut) : une vraie application interrogerait un service de géocodage.

Exemple (Festin Italien : 22 €/pers., minimum 10)	Total
16 personnes à Bordeaux : 352 € – 10 % (35,20 €) + 0 €	316,80 €
12 personnes à Mérignac (8 km) : 264 € + 5 € + 8 × 0,59 €	273,72 €

7.2 Stock

Le stock représente le nombre de commandes encore possibles. Une commande le diminue de 1, une annulation (client ou employé) le rétablit. Un menu épuisé ne peut plus être commandé, contrôle fait côté serveur.

7.3 Cycle de vie d'une commande

en attente → acceptée → en préparation → en cours de livraison → livrée → en attente du retour de matériel → terminée, ou annulée.

- Le client peut modifier (sauf le menu) ou annuler sa commande **tant qu'elle est en attente**.
- Un employé ne peut annuler qu'en indiquant le **mode de contact** (GSM ou email) et le **motif**.
- Chaque changement est horodaté dans `order_status_history` et visible par le client.

7.4 Emails automatiques

Événement	Email
Création de compte	Bienvenue
Mot de passe oublié	Lien de réinitialisation valable 1 heure
Commande validée	Confirmation avec le récapitulatif
Statut « en attente du retour de matériel »	Rappel : restitution sous 10 jours ouvrés, sinon 600 € de frais (CGV)
Statut « terminée »	Invitation à donner un avis
Création d'un compte employé	Notification, sans le mot de passe
Formulaire de contact	Demande transmise à l'entreprise, avec l'adresse du visiteur en « Répondre à »

8. Sécurité

Risque	Mesure mise en place
Vol de mots de passe	Hachage par l'algorithm <code>auto</code> de Symfony (bcrypt ou Argon2) ; aucun mot de passe en clair, ni en base, ni dans les emails.
Mots de passe faibles	10 caractères minimum, avec majuscule, minuscule, chiffre et caractère spécial, règles centralisées dans <code>PasswordPolicy</code> (inscription, réinitialisation, création d'employé).
Accès non autorisé	Hiérarchie de rôles, <code>access_control</code> par préfixe d'URL et <code>#[IsGranted]</code> sur chaque contrôleur ; un client ne peut agir que sur <code>ses</code> commandes. Le rôle est fixé côté serveur à l'inscription.
Compte employé désactivé	<code>UserChecker</code> refuse la connexion d'un compte désactivé, avant même la vérification du mot de passe.
CSRF (requête forgée)	Jeton CSRF sur tous les formulaires, y compris connexion, contact, suppressions et annulations (méthode POST).
Injection SQL	Requêtes Doctrine paramétrées ; aucune concaténation de saisie utilisateur dans une requête.
XSS	Échappement automatique de Twig sur toutes les variables affichées.
Énumération des comptes	Message identique à la demande de réinitialisation, que l'email existe ou non ; jeton aléatoire de 64 caractères, à usage unique, expirant après 1 heure.
Fichiers envoyés (galerie)	Contrôle serveur du type réel (JPG, PNG, WebP) et de la taille (2 Mo) ; nom de fichier aléatoire, jamais celui fourni par l'utilisateur.
Fuite de secrets	Secrets uniquement dans <code>.env.local</code> (ignoré par Git) et dans les variables de Render ; diagnostic de démarrage qui masque les identifiants.
Interception réseau	HTTPS en production ; proxys de confiance configurés pour que Symfony génère des liens en https et des cookies sécurisés (<code>cookie_secure: auto</code> , <code>SameSite=Lax</code>).
Données personnelles (RGPD)	Seules les données nécessaires à la commande sont collectées ; mentions légales et CGV accessibles depuis toutes les pages.

9. Accessibilité (RGAA)

- **Structure** : langue `fr`, titre propre à chaque page, zones `header`, `nav`, `main` et `footer`, hiérarchie des titres respectée.
- **Navigation au clavier** : lien d'évitement « Aller au contenu », contour de focus visible sur tous les éléments interactifs, menus refermables avec Échap, galerie utilisable au clavier.
- **Contrastes** : textes à 4,5:1 minimum (critère 3.2). Plus de 100 couleurs trop claires ont été corrigées.
- **Lecteurs d'écran** : icônes décoratives masquées (`aria-hidden`), textes alternatifs sur les photos, notes des avis annoncées (« Note : 4 sur 5 »), boutons nommés, champs de formulaire reliés à leur étiquette.
- **Liens** : soulignés dans le texte, pour ne pas reposer sur la seule couleur.
- **Affichage sur petit écran** : pas de défilement horizontal ; sous 900 px, les liens se replient derrière un bouton « Menu » (`aria-expanded`).
- **Mouvements** : animations désactivées si l'utilisateur a demandé à les réduire.
- **Mises à jour dynamiques** : le nombre de menus trouvés après filtrage est annoncé (`aria-live`).

10. Déploiement

10.1 Démarche

L'hébergeur devait accepter PHP 8.2, l'extension MongoDB et une connexion vers MongoDB Atlas, gratuitement. Les hébergeurs mutualisés gratuits ne permettent pas d'installer l'extension ou interdisent les connexions sortantes vers une base. D'où le choix d'une **image Docker** qui embarque tout ce qu'il faut, hébergée sur **Render** et reconstruite automatiquement à chaque `git push` sur `main`.

10.2 Fichiers de déploiement

Fichier	Rôle
<code>Dockerfile</code>	Image PHP 8.2 + Apache ; extensions intl, zip, opcache et mongodb ; <code>composer install -no-dev</code> ; compilation des assets.
<code>docker/apache.conf</code>	Racine web sur <code>public/</code> , toutes les URL confiées à Symfony.
<code>docker/php.ini</code>	Réglages de production : erreurs masquées, OPcache, taille des envois.
<code>docker/start.sh</code>	Script de démarrage (voir 10.3).
<code>docker/diagnostic.php</code>	Vérifie le format de <code>MAILER_DSN</code> et la connexion MongoDB, identifiants masqués.
<code>render.yaml</code>	Description du service Render (Docker, offre gratuite, variables attendues).
<code>.dockerignore</code>	Exclut notamment <code>.env.local</code> et <code>vendor/</code> de l'image.

10.3 Étapes

1. **MongoDB Atlas** : créer un utilisateur de base, puis autoriser l'accès réseau `0.0.0.0/0` (l'adresse IP de Render n'est pas fixe ; la base reste protégée par identifiant et mot de passe).
2. **GitHub** : pousser le code, Dockerfile compris, sur la branche `main`.
3. **Render** : créer un *Web Service* relié au dépôt, runtime **Docker**, région Francfort, offre *Free*.
4. **Variables d'environnement** saisies dans Render, jamais dans le code : `APP_ENV=prod`, `APP_SECRET` (générée par Render), `MAILER_DSN` (port 2525, car Render bloque les ports SMTP classiques), `MONGODB_URI`, `MONGODB_DB`. Les valeurs se saisissent **sans guillemets**.
5. **Construction** : Render construit l'image (environ 5 minutes).
6. **Démarrage** (`start.sh`) : création de la base SQLite à partir de `sql/schema.sql` et `sql/data.sql`, diagnostic, migrations éventuelles, synchronisation MongoDB, préchauffage du cache, puis Apache sur le port fourni par Render.
7. **Vérification** : parcours complet sur l'URL publique (pages publiques, connexion, commande, emails reçus dans Mailtrap, tableau de bord alimenté par MongoDB).

Limites de l'offre gratuite : pas de disque persistant, la base repart du jeu de démonstration à chaque redémarrage ; mise en veille après 15 minutes sans visite (environ 1 minute au premier chargement). En production, on utiliserait une base PostgreSQL hébergée et une offre payante.

10.4 Problèmes rencontrés et solutions

Problème	Cause et solution
Pages avec email en erreur 500	Valeur de <code>MAILER_DSN</code> collée avec ses guillemets ; valeur corrigée, puis ajout du script de diagnostic.
« Impossible de joindre MongoDB » au démarrage	Même cause sur <code>MONGODB_URI</code> ; le démarrage continue sans bloquer le site.

10.5 Git et mises à jour

Branches : `main` (production), `develop` (intégration), `feature/...` (une par fonctionnalité). Une fonctionnalité est développée sur sa branche, testée, fusionnée dans `develop`, puis `develop` est fusionnée dans `main`, ce qui redéploie le site automatiquement.

11. Tests et vérifications

Vérification	Résultat attendu et obtenu
Calcul du prix (8 cas : minimum, sous le minimum, seuil de réduction, hors Bordeaux...)	Montants identiques aux calculs manuels (ex. 316,80 € et 273,72 €).
Aperçu du prix avant validation	Égal au montant facturé.
Stock	-1 à la commande, +1 à l'annulation ; menu épuisé refusé même par URL directe.
Droits d'accès	Un client ne peut ni ouvrir l'administration ni annuler la commande d'un autre (erreur 403).
Emails (7 types)	Reçus dans Mailtrap ; un échec d'envoi n'empêche ni l'inscription ni la commande.
Fichiers SQL	Base recréée depuis <code>schema.sql</code> + <code>data.sql</code> : 20 tables identiques à la base de l'application, aucune clé étrangère cassée.
Tableau de bord	5 commandes, 1 096,47 € de chiffre d'affaires, identique à la somme des commandes SQL.
Production	Toutes les pages publiques répondent (code 200) sur l'URL Render.